

Index

Teknisk dokumentation	3
1. Introduktion.....	3
1.1 Projektpresentation	3
1.2 Varför detta system? Problemställning	3
1.3 Systemöversikt: lösningen på problemet	4
1.4 Fil och mappstruktur	4
1.5 Applikationens komponenter	6
1.6 Flöden – översikt och flödesschema.....	7
2. Arbetsmetodik.....	8
2.1 Iterativt arbete i små steg.....	8
2.2 ADR (Architecture Design Records)	8
2.3 Överblick av dokumentationen.....	9
2.4 Journal – documentation under resans gång	10
2.5 Övrig dokumentation	10
2.6 LLM och dess roll i projektet.....	11
2.7 Övrig hjälp och stöd.....	11
3. .NET	12
3.1 .NET 8	12
3.2 Blazor Server	12
3.3 ASP.NET Core Identity	12
3.4 Controller-baserad JSON-API	13
3.5 Dependency Injection.....	13
3.6 Patterns	13
3.7 Error handling Helpers	14
3.8 Navigation Helper.....	14
3.9 Price calculation Helper	15
3.10 Ticket Activation Helper	15
3.11 QR Code Helper	15
3.12 Cosmos JSON Serializer.....	15
3.13 Bibliotek och NuGet-paket.....	16
3.14 TicketStore π: Minimal Session State	16
4. Patterns.....	17
4.1 Outbox Pattern	17

4.2 Sentinel Key Pattern.....	17
4.3 Repository Pattern	18
4.4 Service Layer Pattern	18
4.5 Extension Methods Pattern	18
4.6 Dependency Injection Pattern	19
4.7 Controller-baserad "REST"-Pattern (JSON-Pattern)	19
4.8 Error Handling Pattern	19
4.9 Feature Flag Pattern.....	20
4.10 Dual-System Coexistence Pattern	20
5. Azure.....	20
5.1 App Service	20
5.2 Cosmos DB	21
5.3 Service Bus	21
5.4 Functions.....	21
5.5 Application Insights	22
5.6 App Configuration	22
5.7 Key Vault	22
5.8 Storage Account	23
5.9 Managed Identity och RBAC	23
5.10 Azure Entry ID.....	23
5.11 Kostnader.....	24
6. CI/CD	25
6.1 GitHub Actions.....	25
6.2 Docker	26
6.3 GitHub Container Registry (GHCR)	26
6.4 Bicep (Infrastructure as Code - IaC).....	26
6.5 Autentisering och secrets	27
6.6 Multi-stage Docker builds	27
6.7 Deployment Strategies	27
6.8 Environment Management	28
7. Säkerhet.....	28
7.1 Testning	28
7.2 Autentisering.....	29
7.3 Säkerhet i Azure.....	29
7.4 GDPR och Session Management	30
7.5 Error Handling och Logging	30

8. Lärdomar, utveckling och slutsats	30
8.1 Lärdomar av projektet.....	30
8.2 Hur produkten kan utvecklas	32
8.3 Slutsats.....	33
9. Referenser.....	34
9.1 Microsoft referensmaterial	34

Teknisk dokumentation

1. Introduktion

Detta dokument beskriver den tekniska arkitekturen, designvalen och implementationen av projektet. Källkod finns i det publika GitHub-repot:

<https://github.com/mymh13/clo24-nikhal78-examwork>

Fördjupad dokumentation, inklusive ADR:er, journaler och arkitekturöversikter, återfinns i katalogen docs, med en översikt här:

<https://github.com/mymh13/clo24-nikhal78-examwork/blob/main/docs/README.md>

Dokumentet sammanfattar projektets tekniska lösningar utan att duplicera all detaljerad dokumentation från repot.

1.1 Projektpresentation

Projektet är ett modernt biljettbokningssystem byggt på .NET 8 och Azure, med fokus på att stödja två parallella arkitekturer i realtid:

- Dual-System Coexistence: systemet kan köras synkront eller eventdrivet.
- Runtime-switching med en toggle-switch som styr vilken arkitektur som används.
- Blazor Server + ASP.NET Core API för UI och backend.
- Azure-molntjänster för datalagring, meddelandehantering, telemetri och konfiguration.

Målet är att demonstrera en flexibel, skalbar och praktiskt användbar lösning som möjliggör drift av två system samtidigt.

1.2 Varför detta system? Problemställning

Kundens befintliga IT-miljö består av ett äldre och ett nyare system som båda är i aktiv drift. Det finns därför ett behov av:

- Ett gränssnitt som kan hantera båda systemen parallellt
- En successiv migrering, utan att bryta kompatibilitet med historiska processer
- En lösning som möjliggör växling mellan arkitekturer i runtime, utan service-restart

- Minimal påverkan på befintlig drift och ingen downtime för slutanvändaren

Den framtagna lösningen adresserar detta genom en permanent dual-arkitektur där växling sker via feature flags och konfigurationsstyrning.

1.3 Systemöversikt: lösningen på problemet

Systemet är uppbyggt för att stödja två parallella arkitekturer som kan växlas mellan i realtid: ett synkront flöde och ett eventdrivet flöde. Båda är permanent implementerade och existerar sida vid sida.

Det synkrona systemet använder ett traditionellt API-flöde där bokningar skrivs direkt till Cosmos DB (serverless). Detta ger snabb respons och låg komplexitet.

Det eventdrivna systemet använder ett separerat meddelandeflöde. Outbox Pattern garanterar att händelser skrivs transaktionssäkert och bearbetas oberoende av API-anropet.

Vilket system som är aktivt styrs genom en feature flag i Azure App Configuration. En toggle-knapp i admin-gränssnittet ändrar flaggans värde i runtime, vilket möjliggör omedelbar växling mellan arkitekturerna utan service-restart.

Kort sagt: en permanent dual-arkitektur där toggling, konfigurationsstyrning och Azure-integration löser kundens behov av att driva två system samtidigt.

1.4 Fil och mappstruktur

Projektets struktur är organiserad enligt en tydlig separation mellan dokumentation, applikationskod, infrastruktur och CI/CD-pipelines. Nedan följer en fullständig ASCII-översikt över mappstrukturen med kommentarer för varje del:

```
clo24-nikhal78-examwork/
├─ docs/                                # ALL documentation
│  ├─ README.md                        # Documentation overview
│  ├─ adr/                             # Architecture Decision Records
│  │  ├─ archive/                      # Superseded ADRs
│  │  └─ README.md
│  ├─ glossary.md                     # Terms (zone, trip, booking, etc.)
│  ├─ initial_outtakes/               # Drafts and original ideas
│  │  ├─ architecture.md
│  │  ├─ cosmos-ticketing-considerations.md
│  │  ├─ system_overview.md
│  │  └─ Examensarbete-utkast.pdf
│  ├─ school_related/                 # Education-specific documentation
│  │  ├─ teknisk_dokumentation_clo24nikhal.docx
│  │  ├─ verktygspresentation_clo24nikhal.docx
│  │  └─ journal/                     # Timeline/Log per week
├─ src/                               # Production code
│  ├─ web/                             # Blazor Server (UI + API)
│  │  └─ Ticketing.Web/               # .NET 8 Blazor Server project
```

```

| | | └─ Controllers/           # API Controllers (Auth, Bookings,
Users, Health, FeatureFlag)
| | | └─ Pages/                 # Razor pages (Login, Admin, User,
Inspector, Demo, Health)
| | | └─ Components/           # Reusable Razor components
(BookingTable, BookingManagement)
| | | └─ Services/              # Business logic services (Booking,
User, Outbox, FeatureFlag, Telemetry)
| | | └─ Helpers/              # Utility helpers (Error, Price,
Navigation, QR Code, Ticket Activation)
| | | └─ Extensions/           # Startup configuration extensions
(Service, WebApp, Configuration)
| | |   └─ Authentication/      # Cookie auth and session management
| | |   └─ Shared/              # Shared Razor components (MainLayout)
| | |   └─ wwwroot/             # Static files (CSS, favicon)
| | |   └─ Dockerfile           # Multi-stage Docker build
| | |   └─ Program.cs           # Application entry point
| | └─ api/                    # Controller-based Web API
(Legacy/unused)
| |   └─ Ticketing.Api/         # .NET 8 Web API project
| └─ functions/                # Azure Functions (event-driven)
|   └─ Ticketing.Functions/    # .NET 8 isolated worker Functions
|     └─ Functions/            # Function implementations
(OnBookingCreatedFunction)
| |   └─ Program.cs             # Function app startup
| |   └─ host.json              # Function app configuration (retry,
Logging)
| |     └─ local.settings.json  # Local development settings
| └─ tests/                    # Test projects
|   └─ Ticketing.Web.Tests/    # xUnit test project for Ticketing.Web
|     └─ Helpers/              # Unit tests
(PriceCalculationHelperTests)
| |   └─ Integration/          # Integration tests
| |     └─ BookingLifecycleTests.cs
| |     └─ WebApplicationFactoryFixture.cs
| |     └─ Mocks/              # In-memory mock services (User,
Booking, Outbox, Storage)
|   └─ shared/                 # Shared Libraries
|     └─ Ticketing.Contracts/  # DTOs, Events, Contracts
|       └─ Bookings/           # Booking, BookingDto, TicketStatus
|       └─ Users/              # User
|       └─ Events/             # Event base class, BookingCreated,
BookingCancelled
|     └─ Outbox/               # OutboxEvent, OutboxEventStatus
|     └─ Ticketing.Domain/     # Domain entities (future/placeholder)
└─ infra/                      # Infrastructure as Code (Bicep)
  └─ modules/                  # Reusable Bicep modules

```

```

├── appconfiguration.bicep
├── applicationinsights.bicep
├── appservice.bicep
├── cosmosdb.bicep
├── functionapp.bicep
├── keyvault.bicep
├── servicebus.bicep
├── env/                                # Environment-specific deployments
│   ├── dev/
│   │   ├── main.bicep
│   │   └── main.parameters.json
│   └── README.md
├── .github/                            # GitHub Actions workflows
│   └── workflows/
│       ├── ci-build.yaml               # CI: Build and push Docker images
│       ├── cd-web-dev.yaml             # CD: Deploy Web app to dev
│       └── cd-functions-dev.yaml        # CD: Deploy Functions to dev
├── letsencrypt/                        # Let's Encrypt SSL certificate
management
├── clo24_nikhal78_examwork.sln         # Solution file
├── .gitignore
├── LICENSE
└── README.md                           # This file

```

För att kunna visualisera överblicken bättre, besök gärna följande länk:

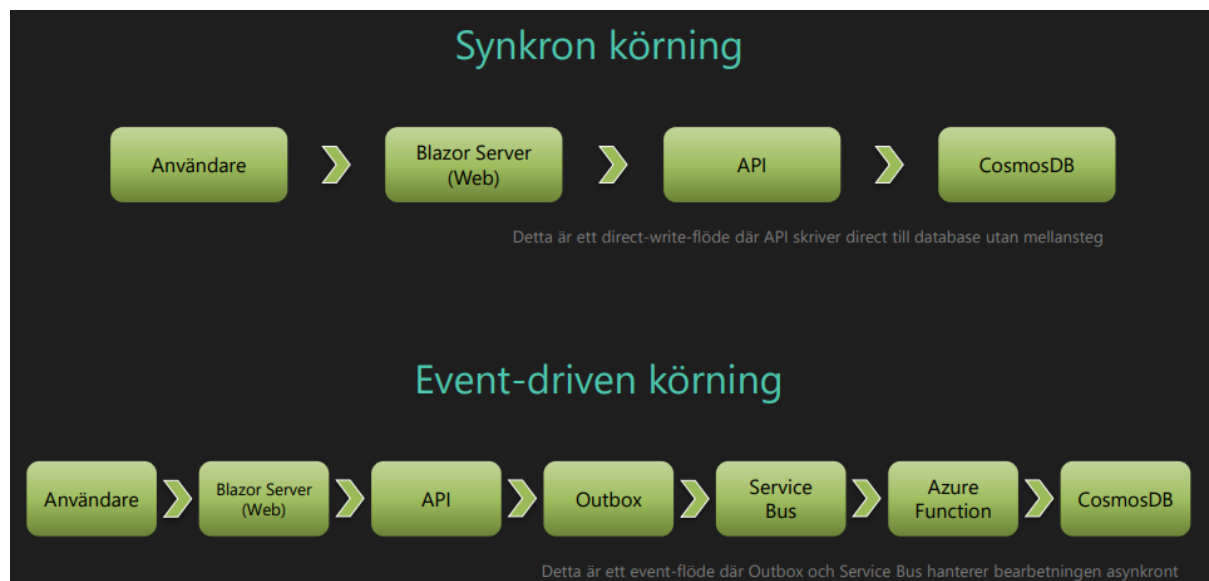
<https://github.com/mymh13/clo24-nikhal78-examwork/blob/main/README.md>

1.5 Applikationens komponenter

Applikationen består av ett antal tydligt separerade komponenter som tillsammans utgör UI, API, affärslogik och hjälpfunktioner. Nedan följer en översikt över de huvudsakliga kategorierna och deras ansvar:

- Controllers: BookingsController, UsersController, AuthController, HealthController, FeatureFlagController
- Services: BookingService, UserService, OutboxService, FeatureFlagService, TelemetryService, EventPublisher
- Helpers: ErrorHandlerHelper, HttpErrorHelper, PriceCalculationHelper, NavigationHelper, TicketActivationHelper, QrCodeHelper
- Pages: Login, AdminLandingPage, UserLandingPage, Bookings, Users, Health, Demo
- Components: BookingTable, BookingManagement
- Extensions: ServiceCollectionExtensions, WebApplicationExtensions, ConfigurationExtensions

1.6 Flöden – översikt och flödesschema



Systemet stödjer två parallella flöden för hantering av bokningar: ett synkront flöde och ett eventdrivet flöde. Vilket flöde som används styrs av en feature flag i Azure App Configuration, som kan ändras i runtime via en toggle-switch i admin-gränssnittet.

1.6.1 Synkront flöde

Användare → Blazor Server → API → Cosmos DB

I det synkrona läget behandlas bokningen direkt av API:et. Anropet skriver omedelbart till Cosmos DB och returnerar sedan ett svar till användaren. Detta flöde är enkelt, snabbt och lätt att förstå.

Fördelar: Snabb respons, lätt att förstå.

Nackdelar: Mindre robust vid hög belastning, mindre transaktionssäkert.

1.6.2 Eventdrivet flöde:

Användare → BlazorS → API → Outbox → Service Bus → Azure Functions → Cosmos DB

I det eventdrivna läget lagrar API:et bokningen som ett Outbox-event. En bakgrundsprocess publicerar eventet till Service Bus, vilket triggar en Azure Function som utför den faktiska bearbetningen och uppdaterar Cosmos DB. Detta ger ett mer flexibelt och skalbart system, särskilt vid hög belastning eller integrationsbehov.

Nyckelkomponenter:

- Toggle-switch / Feature flag: Styr vilket flöde applikationen använder
- Outbox Pattern: Säkerställer transaktionssäkerhet i scenario där API och eventpublicering måste hållas i synk

2. Arbetsmetodik

Projektet har utvecklats med fokus på tydlighet, spårbarhet och kontinuerlig förbättring. Även om ingen formell Scrum- eller sprintstruktur använts har arbetet följt agila principer: små steg, täta iterationer, snabb feedback och löpande refinement av både kod och arkitektur.

Dokumentationen har förts parallellt med utvecklingen, vilket säkerställt att varje större beslut och ändring är motiverad, spårbar och konsekvent genom hela projektets livscykel.

2.1 Iterativt arbete i små steg

Arbetet delades upp i tydliga faser, där varje fas bestod av små, hanterbara delmoment. Detta gav en naturlig arbetsrytm och gjorde det möjligt att utveckla, testa och refaktorera funktionalitet kontinuerligt.

Metoden simulerar ett agilt arbetssätt utan formella sprintar: varje steg byggde vidare på det föregående, och utvecklingen kunde snabbt anpassas vid behov. Varje delmoment validerades lokalt (t ex genom CLI-kommandon eller genom dotnet build) innan de commitades till GitHub.

Varför denna metodik:

Den gav ett stabilt och kontrollerat arbetsflöde där komplex funktionalitet kunde införas successivt utan att tappa överblick eller systemstabilitet. Det underlättade även felsökning. Varför inte agilt fullt ut? Två skäl:

- Dels så har jag arbetat på Öredev och mitt LIA-företag under examensarbetsperioden, det har inte blivit ett jämnt flöde i arbetshastighet
- Dels så arbetar jag själv, så även om jag tidsuppskattat delmoment, så har jag ändå ingen person som står och trampar och väntar på mig

2.2 ADR (Architecture Design Records)

ADR är en metod för att dokumentera viktiga tekniska beslut under projektets gång. Varje ADR beskriver beslutets bakgrund, vilka alternativ som övervägdes, den valda lösningen, samt vilka konsekvenser beslutet får för systemets arkitektur och utveckling.

I projektet skapades 20 ADR-dokument, vilket säkerställer att större arkitekturval är spårbara och motiverade. ADR-formatet gjorde det möjligt att hantera komplexa vägval strukturerat och att kunna lämna efter sig en beslutsmotivering.

Exempel på centrala ADR:

- ADR-001 – Val av Cosmos DB
- ADR-004 – Val av Blazor Server
- ADR-005 – Val av Azure-tjänster
- ADR-013 – Outbox Pattern
- ADR-014 – Sentinel Key Pattern

Varför ADR i projektet:

Det ger långsiktig spårbarhet, tydlig motivering bakom varje designval och en robust grund för både vidareutveckling och framtida beslut.

2.3 Överblick av dokumentationen

Projektet har dokumenterats iterativt och parallellt med utvecklingsarbetet, vilket innebär att beslutslogik, arkitektur och metodik finns nedtecknat i realtid snarare än i efterhand. För att undvika att detta dokument blir överlastat återges inte all detaljerad dokumentation här. I stället bör läsaren bekanta sig med strukturen i docs/-katalogen, som innehåller separata avsnitt för ADR:er, arkitekturmateriel, tidloggar och metodbeskrivningar.

Den bifogade bildöversikten visar huvudstrukturen:

```
## Documentation Structure
docs/
├── README.md # This file - documentation overview
├── glossary.md # Glossary of central concepts and acronyms
├── statistics.md # Project statistics and metrics
├── adr/ # Architecture Decision Records
│   ├── README.md # ADR index and overview
│   ├── ADR-000-template.md # Template for new Architecture Decision Records
│   ├── ADR-001-cosmosdb.md # Decision: Database choice (Azure Cosmos DB, ServerLess)
│   ├── ADR-002-authentication.md # Decision: Authentication via ASP.NET Identity + Entra ID
│   ├── ADR-003-iac.md # Decision: Infrastructure as Code with Bicep
│   ├── ADR-004-frontend.md # Decision: Frontend in .NET 8 Blazor Server
│   ├── ADR-005-azureservices.md # Decision: Core Azure services for operations and monitoring
│   ├── ADR-006-eventdriven.md # Planned decision: Event-driven architecture (Service Bus + Function)
│   ├── ADR-007-ssl-certificate.md # Decision: SSL certificate strategy (manual Let's Encrypt via Docker)
│   ├── ADR-008-docker-deployment.md # Decision: Deployment strategy (Docker containers via GHCR)
│   ├── ADR-009-extension-methods-pattern.md # Decision: Code organization using extension methods pattern
│   ├── ADR-010-gdpr-session-management.md # Decision: GDPR-compliant session management
│   ├── ADR-011-price-calculation-system.md # Decision: Price calculation system with discounts
│   ├── ADR-012-azure-app-configuration.md # Decision: Azure App Configuration for feature flags
│   ├── ADR-013-outbox-pattern.md # Decision: Outbox Pattern for data integrity in dual writes
│   ├── ADR-014-sentinel-key-pattern.md # Decision: Sentinel Key Pattern for hot-reload configuration
│   ├── ADR-015-application-insights-telemetry-strategy.md # Decision: Application Insights custom events for dual-system architecture
│   ├── ADR-016-managed-identity-rbac-strategy.md # Decision: Managed Identity & RBAC for secure Azure service access
│   ├── ADR-017-service-component-organization-pattern.md # Decision: Code organization and separation of concerns
│   ├── ADR-018-error-handling-logging-strategy.md # Decision: Error handling and structured logging strategy
│   ├── ADR-019-api-design-pattern-controller-based-rest.md # Decision: Controller-based REST API design pattern
│   ├── ADR-020-qr-code-implementation.md # Decision: QR code generation and storage strategy
│   └── archive/ # Archive for older or replaced ADR documents (currently empty)
├── initial_outtakes/ # Early drafts and plans that defined the project's direction
│   ├── architecture.md # Architectural overview with ASCII diagrams and flow descriptions
│   ├── cosmos-ticketing-considerations.md # Data and domain considerations for booking and ticket model
│   ├── system_overview.md # System overview with project goals, technical plan, and service choices
│   └── Examsarbete-utkast.pdf # Initial thesis draft (PDF)
├── journal/ # Weekly Logs and project documentation
│   ├── week_one.md # Week 1 - startup, planning, and documentation structure
│   ├── week_two.md # Week 2 - code structure and first runnable version
│   ├── week_three.md # Week 3 - infrastructure and CI/CD foundation
│   ├── week_four.md # Week 4 - infrastructure expansion and application integration
│   ├── week_five.md # Week 5 - feature development and ticket management
│   ├── week_six.md # Week 6 - event-driven architecture refinement & demo enhancements
│   ├── week_seven.md # Week 7 - documentation, presentation and finalization
│   ├── eventdriven_roadmap.md # Detailed roadmap for event-driven architecture refactoring (phases 1-10)
│   ├── phase7_testing_guide.md # Testing guide for Phase 7.1 (synchronous flow validation)
│   ├── phase7_2_testing_guide.md # Testing guide for Phase 7.2 (event-driven flow validation)
│   ├── phase7_3_testing_guide.md # Testing guide for Phase 7.3 (switching between modes)
│   ├── phase7_4_testing_guide.md # Testing guide for Phase 7.4 (additional validation)
│   ├── phase7_validation.md # Test results and validation documentation for Phase 7
│   ├── phase8_dashboard_guide.md # Application Insights Workbook setup guide
│   ├── demo_page_setup.md # Demo page setup and configuration
│   ├── appinsights_troubleshooting.md # Application Insights troubleshooting guide
│   ├── get_appinsights_connection_string.md # Guide for retrieving Application Insights connection string
│   ├── test_activation_endpoint.md # Testing guide for ticket activation endpoint
│   └── app_insights_refresh_query.kql # KQL query for App Configuration refresh activity
├── school_related/ # School-related documents (thesis, presentations)
│   ├── teknisk_dokumentation_clo24nikhal.docx # Technical documentation (Word)
│   ├── teknisk_dokumentation_clo24nikhal.pdf # Technical documentation (PDF)
│   ├── teknisk_template.md # Technical documentation template/structure
│   ├── verktygspresentation_clo24nikhal.docx # Tool presentation (Word)
│   ├── verktygspresentation_clo24nikhal.pdf # Tool presentation (PDF)
│   ├── template_presentation.md # Presentation template/structure
│   ├── presentation_clo24nikhal.pptx # PowerPoint presentation
│   ├── copilot_presentation_guide.md # Copilot in PowerPoint design brief (compressed)
│   ├── copilot_powerpoint_instructions.md # Copilot in PowerPoint instructions
│   └── powerpoint_design_brief.md # PowerPoint design brief
```

- ADR – Architecture Decision Records som dokumenterar alla större tekniska beslut, alternativ och konsekvenser.
- initial_outtakes – Tidiga arkitekturidéer, skisser, utkast och konceptuella modeller.
- journal – Veckovisa loggar som beskriver arbetsprocess, iterationer och valda lösningar.
- school_related – Dokument relaterade till rapportskrivning och presentationer.
- README + glossary + statistics – Översikt, begreppslista och statistiska sammanställningar över projektet.

Varför detta upplägg:

Det möjliggör en hög grad av transparens och spårbarhet, samtidigt som huvudfilen (denna tekniska dokumentation) förblir överskådlig.

2.4 Journal – documentation under resans gång

Projektet har förts fram genom kontinuerlig och veckovis loggning. Journalen dokumenterar utvecklingsförloppet i realtid och innehåller detaljer om genomförda aktiviteter, uppkomna utmaningar och lärdomar vecka för vecka.

Den fungerar både som en tidslinje och som ett arbetsredskap för att spåra arkitekturförändringar, prioriteringar och iterationer. Veckorna 1–6 är dokumenterade i sin helhet, kompletterat med en separat event-driven roadmap som beskriver den mer komplexa refaktoreringen av arkitekturen. Vecka 7 dokumenteras fortfarande i.

Varför journalen är viktig:

Den ger full transparens i arbetsflödet, gör det lätt att förstå varför systemet ser ut som det gör, och stödjer framtida vidareutveckling genom att dokumentera centrala beslutsögonblick. Det är väldigt viktigt att få med en tidslinje.

2.5 Övrig dokumentation

Utöver ADR:er och journaler innehåller docs/ även kompletterande material som stödjer förståelsen av projektet ur både tekniskt och konceptuellt perspektiv:

- Glossary – förklarar centrala begrepp och domän termer
- Systemöversikt – ger en sammanfattande bild av arkitektur, dataflöden och komponenter
- architecture.md – innehåller arkitekturdiagram, ASCII-flöden och tidiga designbeskrivningar
- Bug backlog – listar kända buggar och planerade lösningar
- Future improvements – beskriver potentiella vidareutvecklingsområden och tekniska förbättringar

Syftet med denna struktur:

Att hålla systemets kunskap samlad, spårbar och lättåtkomlig utan att belasta huvuddokumentet med detaljer.

2.6 LLM och dess roll i projektet

LLM-stöd (Large Language Models) har använts som en integrerad del av arbetsprocessen. Modellen har bistått i dokumentation, arkitekturresonemang och kodgenerering, men alltid under tydlig mänsklig kontroll. Stödet har omfattat tre huvudområden:

Dokumentation under kodning:

LLM har använts för att formulera och strukturera dokumentation parallellt med utvecklingen, vilket minskade kontextbyten och säkerställde att beslut dokumenterades direkt. Jag har ex. bett agenten i Cursor "dokumentera det vi gjorde nyss i dokument X".

Arkitektur- och designstöd:

Både prompt-modeller som ChatGPT 5 och Claude Sonnet 4.5, samt agent-mode i Cursor har fungerat som diskussionspartner vid utvärdering av designmönster, alternativ och konsekvenser, särskilt i samband med ADR-skrivandet.

Kodgenerering och assistans:

LLM har ibland genererat kodförslag, testfall och helper-funktioner. Däremot har all kod granskats, justerats och validerats manuellt för att säkerställa korrekthet och kvalitet. Det syns framförallt i delar av koden: jag hade tänkt refaktorera ner stora filer till mindre och mer tydliga segment, jag lät LLMen generera kod i filer ibland så de blev för långa.

Varför LLM-stöd i projektet:

Det har effektiviserat arbetsflödet, förbättrat dokumentationskvaliteten och stöttat komplexa designbeslut utan att ersätta kritiskt tänkande eller manuell implementation. Arkitekturen, problemställningen, presentationen, designen, allting strukturellt har varit mina beslut. Framförallt känner jag att LLMen gjort den löpande dokumentationen lättare, och den har hjälpt mig att testa tekniker jag inte provat tidigare.

2.7 Övrig hjälp och stöd

Här vill jag lämna ett speciellt tack och nämna några som betytt en hel del för detta projekt:

Min fokus-grupp på Campus Mölndal. Dennis, Fredrik, Megan, Nor (alfabetisk, ingen inbördes ordning)! Ni är fantastiska på att lyfta mig och att se varandra, utan er hade detta arbete blivit mycket mer ensamt.

Min handledare på LIAn hos [The Tribe](#) – [Henrik Jönsson](#) – han har gett mig flera konkreta tips, bland annat om Sentinel Key Pattern

Den fantastiska läraren – [Tore Nestenius](#) – vars föreläsningar och bloggar inspirerar, bland annat lånade jag blogposten om [TicketStore och cookie-hantering](#) till detta projekt

Tips från mitt nätverk – [Mats Alritzson](#) – har kommit med flera konkreta tips, nu har jag inte testat/agerat på dem pga primärt tidsbrist, men jag uppskattar engagemanget i mitt examensarbete och de generösa tipsen!

Borde inte ovanstående vara ett slutord? Absolut inte! Den här sektionen i dokumenteringen handlar om verktyg och hjälpmedel, och då är det på sin plats att ovanstående nämns!

3. .NET

Detta projektet använder .NET 8, Microsofts plattform för att bygga fullskaliga applikationslösningar. Plattformen erbjuder ett enhetligt ekosystem där backend, frontend, testning, konfiguration och säkerhet kan byggas med samma språk och samma infrastruktur.

Detta ger minskad komplexitet, förbättrad underhållbarhet (i synnerhet som kunden idag jobbar med .NET-verktygen), och i just det här fallet är det även lättare att integrera ny kod in i gamla strukturer.

3.1 .NET 8

Varför .NET 8 nu när .NET 10 finns ute?

Kunden kör redan .NET 8 i sin miljö idag. Det är ändå ett relativt modernt system med LTS-support. Bra stöd i Azure-tjänsterna och ett gemensamt språk (C#) för både frontend och backend. Det är ett tekniskt modernt, hållbart och ekonomiskt fördelaktigt val för kunden.

3.2 Blazor Server

Blazor Server är det UI-ramverk som används för att bygga gränssnittet i projektet. Det är serverdrivet och bygger på SignalR för realtidskommunikation, vilket gör att gränssnittet kan uppdateras direkt utan sidladdningar.

Blazor använder Razor Components för att strukturera UI:t i tydliga och återanvändbara moduler, och kräver inget JavaScript för kärnfunktionaliteten. Det ger en snabb och lättunderhållen lösning där både frontend och backend delar samma språk (C#).

Varför Blazor Server i detta projekt:

Det används redan i kundens befintliga .NET-miljö, är lätt att integrera med API:erna, kräver minimalt med externa beroenden och erbjuder hög utvecklingshastighet vid iterativt arbete. Men rent krasst, det var en av de techstackarna som fanns med i kravspecen. Jag kan personligen tycka att HTMX hade passat för den här typen av applikation.

3.3 ASP.NET Core Identity

Projektet använder ASP.NET Core Identity för användarhantering och autentisering. Identity tillhandahåller ett stabilt ramverk för inloggning, lösenordshantering och rollbaserad åtkomstkontroll. Systemet använder cookie-baserad autentisering för vanliga användare, kombinerat med BCrypt för säker lösenordshashning.

Rollerna i projektet – Admin, Inspector och User – styr vilka funktioner varje användare får tillgång till, och säkerställer en tydlig behörighetsmodell.

Varför Identity i detta projekt:

Det är en beprövad säkerhetslösning, fullt integrerad i .NET-ekosystemet och kompatibel med både kundens miljö och Azure-tjänsterna. Det ger en grundläggande säkerhet utan behov av externa system eller egen implementation. Huvudskälet var även att det var

otroligt lätt att integrera, när EntraID satt för Admin-rollen så behövdes en smidig men ändå tydlig säkerhetslösning för Inspector och Users login.

3.4 Controller-baserad JSON-API

Projektet exponerar sitt backend genom ett controller-baserat JSON-API i ASP.NET Core. Designen följer principerna för REST-liknande API:er: resurser hanteras via standardiserade HTTP-metoder (GET, POST, PUT, DELETE), och data utbyts i JSON-format.

API-et består av flera separata controllers, bland annat BookingsController, UsersController, AuthController, HealthController och FeatureFlagController, vilket ger en tydlig ansvarsfördelning och enkel testbarhet.

Felhantering är centraliserad genom *ErrorHandlerHelper*, som säkerställer konsekventa HTTP-svar och strukturerad felinformation.

Varför denna design i projektet:

Den är enkel att implementera, lätt att förstå, följer etablerad .NET-praxis och passar projektets behov av tydliga endpoints för UI, adminfunktionalitet och eventdrivna processer. Jag valde även denna design delvis för att vissa av Controllersarna är återanvändbara i andra roller (Inspector har vissa rättigheter som Admin har, exempelvis).

3.4.1 Om REST vs. JSON-API (kort sidonot):

Systemet använder JSON-baserad kommunikation i vad som populärt men felaktigt kallas "REST"-API. För att bättre förstå skillnaden mellan REST, "REST-lik" API och varför vi egentligen borde kalla det vi kallar "REST-API" för JSON-API, se [REST explained for beginners](#).

3.5 Dependency Injection

Projektet använder .NET:s inbyggda Dependency Injection (DI) för att strukturera och organisera applikationens beroenden. DI-containern hanterar instansiering av tjänster och gör det möjligt att registrera dem med olika livscykler (Scoped, Singleton, Transient) beroende på behov.

All funktionalitet exponeras via interface-baserad design, vilket ger löskoppling, förbättrar testbarheten och undviker täta eller onödiga beroenden mellan komponenter. Registrering av tjänster sker genom tydliga extension methods, vilket är min favorit-goto-pattern om Program.cs börjar växa sig för lång.. det är ett högst personligt val för att få en tydligare överblick av körfilen!

Varför DI i detta projekt:

Det är standard i modern .NET-utveckling, underlättar testning, möjliggör enkel mockning och bidrar till en ren arkitektur där ansvar är tydligt separerat.

3.6 Patterns

Applikationen använder flera etablerade arkitekturmönster för att skapa en tydlig, testbar och modulär struktur. Nedan listas de patterns som används i projektet, med hänvisning till respektive avsnitt där de beskrivs mer ingående:

- [Repository Pattern](#) – se avsnitt 4.3
- [Service Layer Pattern](#) – se avsnitt 4.4
- [Extension Methods Pattern](#) – se avsnitt 4.5
- [Dependency Injection Pattern](#) – se avsnitt 4.6
- [Controller-baserad REST/JSON Pattern](#) – se avsnitt 4.7
- [Error Handling Pattern](#) – se avsnitt 4.8
- [Feature Flag Pattern](#) – se avsnitt 4.9
- [Dual-System Coexistence Pattern](#) – se avsnitt 4.10
- [Managed Identity Pattern](#) – se avsnitt 4.11
- [Telemetry Pattern](#) – se avsnitt 4.12

Syfte med patterns:

De skapar en enhetlig arkitektur, minskar komplexitet och underlättar både testning och vidareutveckling. Detaljer och tekniska motiv presenteras i respektive avsnitt. Patterns ger också en tydlighet för utvecklare som vill sätta sig in i kodbasen, kan de patternen så kommer de snabbare förstå kodbasen.

3.7 Error handling Helpers

Projektet använder två hjälpbibliotek för att hantera fel på ett strukturerat och konsekvent sätt:

- ErrorHandlerHelper – centraliserad felhantering för API-controllern, ansvarar för att generera standardiserade HTTP-svar och loggposter.
- HttpErrorHandler – motsvarande hantering för Razor Pages, med fokus på användarvänliga felmeddelanden och korrekt återkoppling i UI:t.

Dessa helpers säkerställer att fel presenteras på ett enhetligt sätt, att ingen känslig information läcker ut, och att all viktig information loggas strukturerat för analys i Application Insights.

Varför detta i projektet:

Det förenklar felsökning, förbättrar användarupplevelsen, bidrar till en robust driftmiljö. Jag implementerade de här hjälpsystemen när jag insåg att jag började upprepa error handling i olika kodfiler, att centralisera hanteringen var ett naturligt val då.

3.8 Navigation Helper

NavigationHelper ansvarar för den rollbaserade navigeringen i applikationen. Den centrala funktionen GetLandingPageUrl() returnerar rätt landningssida baserat på användarens roll, exempelvis Admin, Inspector eller User.

Hjälparen används primärt i *Login.razor* men återkommer även i andra sidor där applikationen behöver styra användaren till korrekt vy efter inloggning eller vid rollbyte.

Varför detta i projektet:

Det ger en tydlig, centraliserad logik som säkerställer konsekvent navigationsbeteende och

minskar duplicering i UI-komponenterna. Den kom till när jag ville skapa ett system för att återkomma till din roll-baserade landing page, men inte ville duplicera kod för varje roll.

3.9 Price calculation Helper

PriceCalculationHelper ansvarar för all logik kopplad till biljettprissättning. Den hanterar zonbaserad prissättning, åldersbaserade rabatter (barn, vuxen, pensionär), studentrabatt samt ett konfigurerbart baspris per zon.

Genom att samla prissättningsreglerna i en central helper hålls både UI och tjänstelager fria från duplicerad logik, vilket gör förändringar och tester betydligt enklare.

Varför detta i projektet:

Det säkerställer konsekventa prisberäkningar över hela systemet och ger en strukturerad plats för framtida ändringar i prissättning och affärsregler. Men skälet till att denna blev till var helt enkelt för att inte ha ett hårdkodat pris i koden. Med detta verktyg blir det lättare att manipulera kostnaden och bli mer dynamisk, och/eller prisjustera.

3.10 Ticket Activation Helper

TicketActivationHelper hanterar logiken kring aktivering av biljetter. Den validerar om en biljett kan aktiveras, beräknar giltighetsperioden (90 minuter) och utför statuskontroller mot biljettens livscykel: Created, Activated, Valid och Expired.

Kärnfunktionerna *CanActivate()* och *ValidateActivation()* används av både UI och tjänstelager för att säkerställa att aktivering sker korrekt och enligt regelverket.

Varför detta i projektet:

Det centraliserar all aktiveringslogik, minskar duplicering och gör systemet lättare att testa samt enklare att vidareutveckla vid förändrade affärsregler.

3.11 QR Code Helper

QrCodeHelper använder QRCode-biblioteket för att generera QR-koder kopplade till biljetter. QR-koderna skapas som PNG-bilder, kodas till Base64-strängar och lagras i Cosmos DB tillsammans med JSON-strukturerad data (bland annat Booking ID, Customer ID, tidsstämplar och status).

Funktionen *GetQrCodeDataUrl()* används för att exponera QR-koden direkt i UI:t via en data:-URL, utan behov av separat filhantering.

Varför detta i projektet:

Det ger en enkel lösning för att koppla varje biljett till en unik, maskinläsbar identifierare som är lätt att visa i gränssnittet och möjlig att validera om biljettinspektören implementeras.

3.12 Cosmos JSON Serializer

Projektet använder en anpassad JSON-serializer för Cosmos DB för att hantera de detaljer och begränsningar som följer med databasen, exempelvis property-namn, typer och

systemfält. Lösningen bygger på System.Text.Json, vilket ger hög prestanda och god kontroll över serialiseringsprocessen.

Denna serializer säkerställer att dokumentformatet är konsekvent, kompatibelt med Cosmos DB:s krav och tillförlitligt vid både läsning och skrivning.

Varför detta i projektet:

Det eliminerar serialiseringsproblem och skapar en stabil datamodell som är enkel att felsöka och vidareutveckla. Jag fick problem med serialiseringen av enums och då föddes den här lösningen. Jag har noterat i dokumentationen att enum har sina styrkor, men även medför risker just när det gäller serialisering. Här fick jag värdefulla tips av min handledare på LIAn.

3.13 Bibliotek och NuGet-paket

Projektet använder ett antal centrala NuGet-paket och bibliotek som stödjer funktionalitet, konfiguration och integrationer:

- QRCode – generering av QR-koder för biljetter
- System.Text.Json – snabb och flexibel JSON-serialisering
- Microsoft.Extensions.* – kärnbibliotek för konfiguration, dependency injection och logging
- Azure.* – officiella SDK-paket för Cosmos DB, Service Bus, App Configuration och Key Vault

Tillsammans ger dessa paket en modern, väl underhållen/supportad och fullt integrerad teknisk bas för applikationen.

3.14 TicketStore π: Minimal Session State

Projektet använder en lättviktig modell för sessions- och cookiehantering baserad på [Tore Nestenius'](#) rekommendationer i ["Improving ASP.NET Core Security by Putting Your Cookies on a Diet"](#).

Syftet är att minimera mängden data som lagras i cookies, förbättra säkerheten och undvika onödig exponering av sessionstillstånd i klienten. Lösningen bygger på:

- Server-side session storage (TicketStore) – endast en referens (session-ID) lagras i cookien, medan all sessiondata lagras på serversidan.
- Minimal cookie payload – ingen känslig eller omfattande data skickas till klienten.
- Cookie-fabrik (ISessionFactory) – centraliserad generering av säkra, signerade cookies.
- Förbättrad säkerhet – mindre attackyta, enklare hantering av sessionexpiration och rollinformation.
- Kompatibelt med GDPR-strategin (ADR-010) – eftersom sessiondata aldrig lämnar servern.

Varför detta i projektet:

Det ger en säker, modern och lättviktig sessionhantering, reducerar klientexponering och

följer etablerade säkerhetsprinciper som dataminimering och least privilege. Plus en bonus: Det är kul att prova nya tekniska lösningar, för att utvecklas.

4. Patterns

Applikationen använder flera etablerade arkitekturmönster för att skapa en tydlig struktur. Varje pattern adresserar ett specifikt problem. Vinsten med att använda dem blir både i form av att koden blir lättare att läsa, men hjälper även till om kodbasen skalar upp i storlek.

Nedan följer de patterns som använts i projektet, tillsammans med en kort beskrivning av vad de löser och varför de valts. Mer djupgående beskrivningar finns i respektive underrubrik.

4.1 Outbox Pattern

Outbox Pattern används för att garantera att databasoperationer och eventpublicering sker på ett säkert och konsekvent sätt, särskilt när samma händelse behöver skrivas både till Cosmos DB och publiceras till Service Bus. Utan detta uppstår det klassiska *dual write*-problemet, där en av operationerna kan lyckas medan den andra misslyckas.

I projektet implementeras Outbox Pattern genom:

- En outbox-container i Cosmos DB där events lagras med status (Pending, Processed, Failed).
- Alla bokningar skapar ett event atomiskt tillsammans med databasskrivningen.
- En bakgrundstjänst (OutboxProcessorService) pollar kontinuerligt outboxen och publicerar events till Service Bus.
- Retry-logik med exponential backoff säkerställer att tillfälliga fel hanteras robust.
- Audit trail bibehålls genom att alla events sparas oavsett utfall.

I det synkrona systemet skapas eventet men publiceras inte.

I det eventdrivna systemet publiceras eventet till Service Bus, som sedan triggar Azure Functions för vidare bearbetning.

Varför Outbox Pattern i detta projekt:

Det säkerställer konsekvent databehandling i en arkitektur där både synkrona och eventdrivna flöden måste fungera parallellt, utan risk för inkonsistens mellan databasen och kö-systemet.

4.2 Sentinel Key Pattern

Sentinel Key Pattern är centralt i projektet eftersom systemet ska kunna växla mellan synkront och eventdrivet läge i realtid, utan driftstopp.

I detta projekt implementeras mönstret genom:

- En särskild trigger-nyckel, Settings:Sentinel, i Azure App Configuration.

- När denna nyckel ändras uppdateras alla konfigurationsvärden automatiskt.
- Ett refresh-intervall på 30 sekunder, kompletterat med middleware som anropar `TryRefreshAsync()` vid varje HTTP-request.
- Full zero-downtime switching, vilket innebär att applikationen omedelbart kan ta nya inställningar i bruk.

Varför Sentinel Key Pattern i detta projekt:

Det möjliggör den centrala funktionaliteten i lösningen: runtime-växling mellan två systemarkitekturer utan avbrott. Mönstret är dessutom avgörande för demo-scenariot där systemet ska uppdateras live inför publik.

Vill notera att jag hade initialt tänkt att köra Feature Flags, men kände inte till Sentinel Key så då hade jag förväntat mig att få starta om servicen mellan varje toggle. Jag pratade med min handledare på LIAn, [Henrik Jönsson](#), och han tipsade om Sentinel Key. Detta är första gången jag implementerar en sådan lösning.

4.3 Repository Pattern

Repository Pattern används för att abstrahera dataåtkomst och skapa en tydlig separation mellan applikationslogik och datalager. I projektet definieras gränssnitt som `IBookingService`, `IUserService` och `IOutboxService`, vilket gör det möjligt att byta datakälla eller implementering utan att resten av systemet påverkas.

Mönstret förenklar även testning, eftersom mock- och in-memory-implementationer kan användas utan beroenden till Cosmos DB eller andra externa system.

Varför detta pattern i projektet:

Det ger renare arkitektur, bättre testbarhet och minskar kopplingen mellan affärslogik och dataåtkomst.

4.4 Service Layer Pattern

Service Layer Pattern strukturerar applikationens affärslogik i separata serviceklasser, vilket gör att controllers kan fokusera på HTTP-hantering och input/output-modeller. Den komplexa logiken ligger i services såsom `BookingService`, `UserService` och `OutboxService`, vilket skapar en tydlig ansvarsfördelning och förbättrar läsbarheten i hela koden.

Notera symbiosen det skapar med Repository Pattern ovan. Isolerad affärslogik. Säkrar data.

Varför detta pattern i projektet:

Det möjliggör en modulär och utbyggbar arkitektur, förenklar felsökning och gör det enklare att testa affärslogik isolerat från UI och controllers.

4.5 Extension Methods Pattern

Extension Methods Pattern används för att organisera applikationens startup- och konfigurationslogik i separata, återanvändbara metoder. I projektet delas detta upp i exempelvis `ServiceCollectionExtensions`, `WebApplicationExtensions` och `ConfigurationExtensions`, vilket skapar en mer strukturerad och läsbar *Program.cs*.

Genom att samla registreringar och konfigurationssteg i tydligt namngivna extension methods får projektet en modulariserad uppstart med hög återanvändbarhet.

Det här är ett favorit-pattern hos mig, det gör att Program.cs blir lättare att läsa, vi ser tydligt körordning (vilket kan vara relevant ibland), vi ser direkt vilka services som är kopplade till vilka funktioner, etc.

Varför detta pattern i projektet:

Det ger en renare kodbas, enklare navigation och minskar komplexitet i applikationens startsekvens. Det löser inget problem för kunden, men det gör utvecklarens jobb lättare.

4.6 Dependency Injection Pattern

Dependency Injection Pattern är centralt i .NET och används genomgående i projektet. Genom interface-baserad design och constructor injection kan tjänster med olika livscykler (Scoped, Singleton, Transient) registreras och tilldelas på ett kontrollerat sätt.

Patternet kompletterar [Repository](#)- och [Service Layer](#)-mönstren (se 4.3 och 4.4) och gör det möjligt att ersätta implementationer i testmiljöer samt minska kopplingen mellan komponenter.

Varför detta pattern i projektet:

Det ger en flexibel arkitektur som är lätt att testa, underhålla och vidareutveckla, samtidigt som det följer etablerade .NET-principer.

4.7 Controller-baserad "REST"-Pattern (JSON-Pattern)

Detta pattern används för att strukturera API:et enligt välkända [så kallade REST](#)-principer, där resurser exponeras genom standardiserade HTTP-metoder (GET, POST, PUT, DELETE) och data utbyts i JSON-format.

Applikationen följer en konsekvent URL-struktur och använder HTTP-statuskoder för att signalera utfall vid varje operation, vilket förenklar både implementation och felsökning.

Varför detta pattern i projektet:

Det ger en tydlig, förutsägbar API-design som passar .NET:s controller-modell och fungerar väl tillsammans med både Blazor Server och externa integratörer.

4.8 Error Handling Pattern

Error Handling Pattern centraliserar felhantering i applikationen och säkerställer att API och UI svarar på ett enhetligt sätt vid fel.

Projektet använder ErrorHandlerHelper för controllers och HttpErrorHandler för Razor Pages, vilket ger konsekventa felmeddelanden, strukturerad logging och en förbättrad användarupplevelse.

Varför detta pattern i projektet:

Det minskar risken för inkonsekvent felhantering, underlättar felsökning och säkerställer att inga känsliga detaljer läcker ut i felresponsen. Gillade även att samla all felkodshantering under ett tak, det blir lättare att hitta och hjälper till mot möjlig duplicering av kod.

4.9 Feature Flag Pattern

Feature Flag Pattern används för att aktivera och styra funktionalitet i runtime utan omstart av applikationen. I projektet hanteras detta via Azure App Configuration, där flaggan `BookingEvents_Enabled` avgör om systemet ska köras i synkront eller eventdrivet läge.

Flaggan uppdateras dynamiskt genom [Sentinel Key Pattern](#) (se 4.2), vilket gör att växlingen kan ske direkt. Admin-gränssnittet innehåller en toggle-knapp som ändrar flaggans värde i realtid.

Varför detta pattern i projektet:

Det möjliggör flexibel konfiguration, live-demonstrationer och driftsättning av nya funktioner utan avbrott.

4.10 Dual-System Coexistence Pattern

Dual-System Coexistence Pattern beskriver arkitekturen där två olika systemflöden, i detta fall ett synkront och ett eventdrivet, kan köras parallellt och växlas mellan i realtid.

Feature-flaggan styr aktivt system, men båda flödena existerar och fungerar sida vid sida. Outbox är alltid aktiv och loggar event oavsett vilket system som används. Den synkrona modellen fortsätter fungera oförändrad, vilket gör att inga brytande ändringar införs vid växling.

Varför detta pattern i projektet:

Det möjliggör en permanent arkitektur där både äldre och nyare systemlogik kan samexistera, vilket är nödvändigt för kundens successiva migreringsbehov.

5. Azure

Azure utgör molnplattformen för hela applikationen och ansvarar för hosting, databashantering, meddelandeflöden, konfigurationsstyrning och säker åtkomstkontroll. Projektet använder en uppsättning integrerade Azure-tjänster som tillsammans möjliggör den duala systemarkitekturen (synkront + eventdrivet), säker drift, hög flexibilitet och kostnadseffektiv utveckling.

Varje tjänst fyller en specifik funktion och är antingen vald för att den redan finns i kundens existerande system, eller för att fyller en specifik funktion i det eventdrivna flödet.

5.1 App Service

Azure App Service används som hostingplattform för Blazor Server-applikationen och dess API. Projektet körs i en Basic B1-plan, baserad på Linux med .NET 8-support.

Applikationen distribueras som [Docker](#)-containers direkt [från GitHub Container Registry \(GHCR\)](#), vilket möjliggör förutsägbara deployments och versionshantering.

App Service är dessutom kopplad till Azure via Managed Identity, vilket ger säker åtkomst till Cosmos DB, App Configuration och Service Bus utan att några secrets eller connection strings behöver hanteras i kod.

Varför App Service i projektet:

Det är en stabil, kostnadseffektiv och fullt managerad hostinglösning som passar utmärkt för våra behov i detta projekt.

5.2 Cosmos DB

Cosmos DB används som applikationens primära datalager. Det är en serverless NoSQL-databas, vilket innebär att kostnaden skalar baserat på faktisk användning, inte reserverad kapacitet. Datamodellen består av tre containers: bookings, users och outbox, där outboxen stödjer eventdrivna flöden och auditloggning.

Partition keys används för att möjliggöra effektiv och förutsägbar query-prestanda, och databasen konfigureras med transactional consistency, vilket säkerställer att bokningar och relaterade event skrivs på ett tillförlitligt sätt.

Varför Cosmos DB i projektet:

Den erbjuder låg latency, flexibel datamodellering, serverless billing och direkt integration med Managed Identity, vilket gör den idealisk för ett skalbart biljettsystem. Varför serverless? En vanlig databas med reserverad kapacitet har en fast kostnad. Serverless skalar efter behov, men framför allt, kostar bara vid användning.

5.3 Service Bus

Azure Service Bus används som meddelandekö i det eventdrivna systemflödet. Projektet använder en dedikerad booking-events-kö, där API:t placerar event som senare bearbetas av Functions. Service Bus erbjuder stöd för dead-letter queues, vilket gör det möjligt att hantera felaktiga eller återkommande misslyckade meddelanden utan att blockera huvudflödet.

I utvecklingsmiljö körs tjänsten på Basic-tier, vilket är fullt tillräckligt för projektets belastningsnivå. Åtkomsten hanteras via Managed Identity, vilket eliminerar behovet av secrets.

Varför Service Bus i projektet:

Det är en välintegrerad lösning för att bygga eventdrivna flöden, särskilt i kombination med Outbox Pattern och Azure Functions.

5.4 Functions

Azure Functions används för att bearbeta meddelanden som skickas via Service Bus och därmed realisera applikationens eventdrivna flöde. Funktionen körs som en serverless .NET 8 isolated worker, vilket innebär snabb uppstart, isolerad runtime och enkel paketering via CI/CD.

Functions körs i en Basic B1-plan i utvecklingsmiljön och använder Managed Identity för säker åtkomst till både Cosmos DB och Service Bus utan att connection strings behöver hanteras.

Varför Functions i projektet:

De erbjuder en lättviktig, kostnadseffektiv och välskalande lösning för att hantera händelsebearbetning, med minimal infrastruktur och hög driftsäkerhet. Symbiosen mellan Functions, Service Bus, Outbox Pattern och det eventdrivna är ett huvudskäl.

5.5 Application Insights

Application Insights används för loggning, övervakning och prestandamätning i både App Service och Function App. Genom KQL-queries kan systemets beteende analyseras i detalj, särskilt skillnaderna mellan synkrona och eventdrivna flöden.

Projektet har ett abstraherat lager för telemetri via `ITelemetryService`, vilket möjliggör strukturerade custom events såsom *BookingCreated*, *OutboxEventCreated*, *OutboxEventProcessed*, *ServiceBusEventPublished*, *FeatureFlagToggled* och *ModeSwitch*. Dessa kompletteras med custom dimensions som `SystemType`, `ToMode` och `ArchitectureMode` för att kunna särskilja och jämföra olika körlägen.

Visualisering sker via Insights Workbooks, bl.a. *Current Mode Indicator*, *Latest Booking Flow Timeline* och *Events by Type*. Tjänsten körs i pay-as-you-go-läge, vilket är kostnadseffektivt för utvecklingsmiljö.

Varför Application Insights i projektet:

Det ger full insyn i systemets beteende, möjliggör tydlig jämförelse mellan arkitekturlägen och är avgörande för felsökning och demonstrationer.

5.6 App Configuration

Azure App Configuration används för centraliserad hantering av konfigurationsvärden och feature flags. Den viktigaste flaggan är `BookingEvents_Enabled`, som styr om applikationen körs i synkront eller eventdrivet läge.

Tjänsten stödjer hot-reload via [Sentinel Key](#) Pattern (se 4.2), vilket innebär att ändringar i konfigurationen kan slå igenom utan omstart av applikationen. Den körs på Free tier i utveckling och åtkomst sker via Managed Identity, vilket eliminerar behovet av lagrade secrets.

Varför App Configuration i projektet:

Det är en säker, centraliserad och lättintegrerad lösning för runtime-styrning av systemets arkitektur och beteende.

5.7 Key Vault

Azure Key Vault används för säker lagring av secrets och anslutningsinformation. Tjänsten körs i Standard tier och används i projektet som en central, RBAC-styrd plats för känsliga uppgifter som inte får förekomma i kod eller konfigurationsfiler.

Åtkomst sker via Managed Identity, vilket säkerställer att endast behöriga applikationskomponenter kan läsa nödvändig information utan att hantera lösenord eller connection strings.

Varför Key Vault i projektet:

Det ger en säker och anpassad lösning för hantering av känslig information och det följer Microsofts rekommenderade praxis. Den gör det även den interna accessen oerhört smidig, jag ger olika tjänster olika rättigheter och sedan kan de få all information de behöver genom automatisering.

5.8 Storage Account

Azure Storage Account används av Function App som en obligatorisk komponent för runtime-hantering. Det ansvarar för state management, checkpoints, triggers och olika metadata som behövs för Function Appens interna drift.

I projektet används ett Standard LRS-konto, vilket är tillräckligt för utvecklingsmiljö och erbjuder kostnadseffektiv datalagring. Storage Account används inte direkt av applikationen, utan fungerar som en infrastrukturnivå beroende till Functions.

Datalagringen är persistent, det liknar mer en dedikerad mini-hårddisk än RAM eller cache.

Varför Storage Account i projektet:

Det är en nödvändig del av Azure Functions-arkitekturen och kräver minimalt med administration, men är avgörande för korrekt drift.

5.9 Managed Identity och RBAC

Projektet använder Azure Managed Identity för att alla applikationskomponenter ska kunna autentisera sig mot Azure-tjänster utan lagrade secrets eller connection strings. Åtkomsten styrs helt via RBAC (Role-Based Access Control), vilket ger finmaskig och spårbar åtkomstkontroll.

Roller i projektet inkluderar bland annat:

- App Service: *App Configuration Data Reader, Azure Service Bus Data Owner*
- Function App: *Azure Service Bus Data Receiver, DocumentDB Account Contributor*

Genom att tilldela dessa roller direkt till respektive Managed Identity kan tjänsterna kommunicera säkert med App Configuration, Service Bus och Cosmos DB.

Varför Managed Identity i projektet:

Det ger en säker, automatiserad och underhållsfri autentiseringslösning som följer praxis och eliminerar behovet av connection strings i kodbasen.

5.10 Azure Entry ID

Azure Entra ID används för autentisering av Admin- (och är påtänkt för Inspector-) -roller i projektet. Användare loggar in via sina Microsoft-konton, vilket ger en säker och väl etablerad identitetslösning byggd på OAuth 2.0 och OpenID Connect.

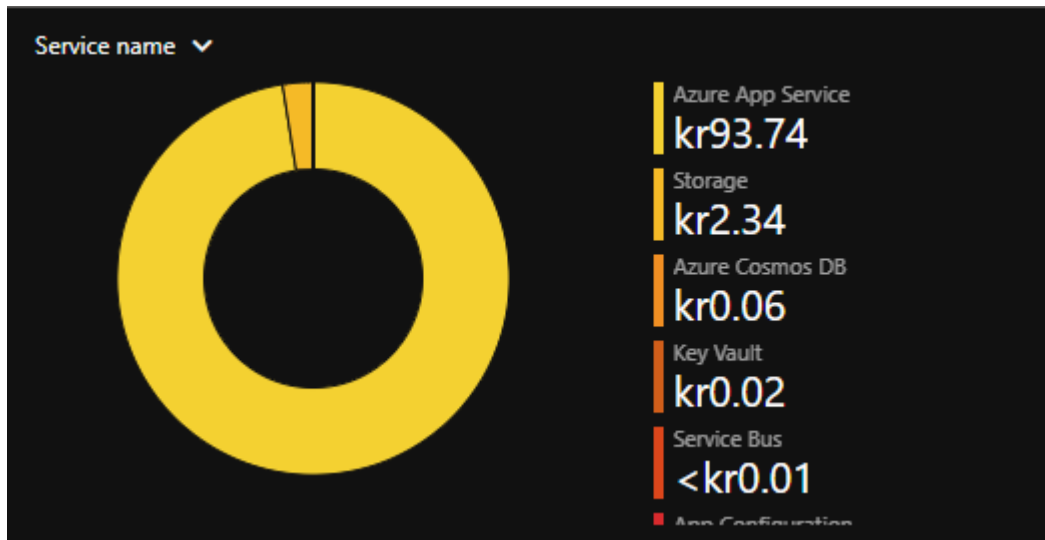
Denna integration ger en robust inloggningsprocess med minimalt underhåll och utan behov av ett eget internauth-system för administratörsroller.

Varför Entra ID i projektet:

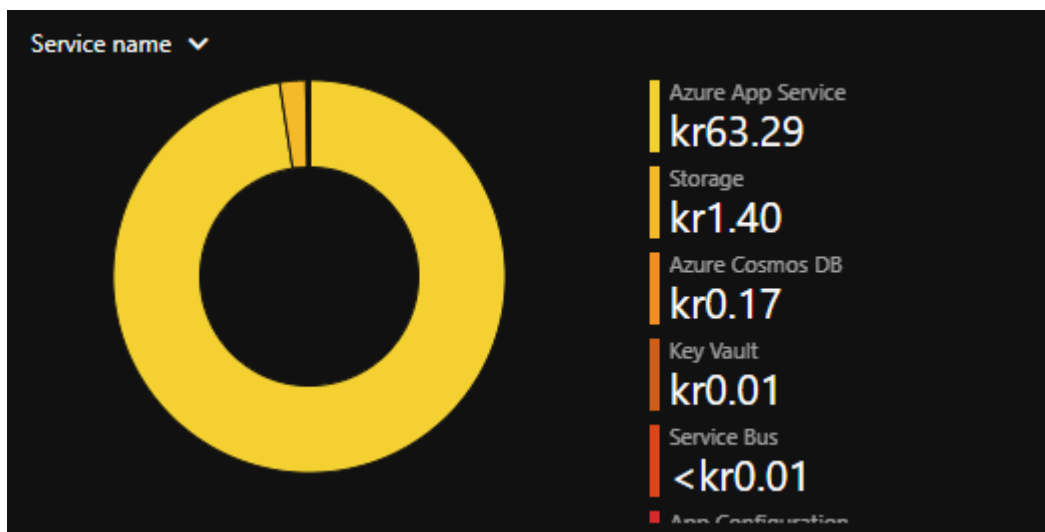
Det ger en trygg och standardiserad autentisering som integrerar väl med Azure-ekosystemet och separerar privilegierade roller från vanliga users. Enkel implementation.

5.11 Kostnader

Driftskostnader november:



Driftskostnader december (fram till 9e då detta dokument skrevs):



Projektets kostnadsbild är tydlig och stabil över tid. De två graferna (november respektive december) visar hur driftskostnaderna fördelas mellan Azure-tjänsterna i utvecklingsmiljön. Den enskilt största kostnaden är Azure App Service, som står för merparten av månadskostnaden. Detta är förväntat eftersom App Service är den enda tjänst som körs kontinuerligt och debiteras per timme även vid låg belastning.

Övriga tjänster: Storage Account, Cosmos DB, Key Vault, Service Bus och App Configuration, ligger mycket lågt i kostnad, samtliga under några kronor per månad. Dessa tjänster faktureras huvudsakligen baserat på faktisk användning (serverless eller transaktionsbaserat), vilket gör deras kostnad marginell i ett projekt av denna storlek.

I både november och december syns samma mönster:

- App Service: ~63–94 kr (dominerande kostnad)
- Storage: ~1–2 kr
- Cosmos DB: ~0.06–0.17 kr
- Key Vault: ~0.01–0.02 kr
- Service Bus: <0.01 kr
- App Configuration: ingår i free tier

Denna kostnadsprofil visar att lösningen är kostnadseffektiv i serverless-delarna, medan App Service står för den huvudsakliga driftskostnaden på grund av sin always-on-modell. I en framtida produktionssättning skulle autoskalning eller alternativ hosting kunna övervägas beroende på användningsfall.

Varför detta är relevant:

Det ger en transparent bild av kostnadsfördelningen och visar att den eventdrivna arkitekturen är billig att underhålla, med App Service som den enda betydande kostnadsposten. App Service har även tydliga alternativ om vi körde större skala: exempelvis VM eller kanske till och med en hybrid-lösning med en on-prem-server.

6. CI/CD

Projektet använder en grundläggande men komplett CI/CD-kedja baserad på GitHub Actions, Docker och Azure. Syftet är att automatisera bygg-, test- och deployprocesserna, minska manuell hantering och säkerställa konsekventa leveranser. Arkitekturen är uppdelad i tre separata workflows: ett för build (CI) och två för deploy (CD): En för Web, en för Function.

Autentisering mot Azure sker via en hybridlösning med OIDC permissions och GitHub Secrets. Workflows använder OIDC (id-token: write) för säker åtkomst, medan Azure-autentiseringsuppgifter (client ID, tenant ID, subscription ID) och Application Insights connection string lagras som GitHub Secrets för säker hantering. Deploymentflödet använder environment protection för att skilja utvecklingsmiljö från produktionsmiljö och säkerställa manuell kontroll där det behövs.

Lösningen är enkel, repeterbar och kostnadseffektiv för både utveckling och drift.

6.1 GitHub Actions

GitHub Actions fungerar som projektets CI/CD-plattform och hanterar automatisering av bygg-, test- och deploysteg. Pipelines definieras i tre YAML-filer: ci-build.yaml för build och push till GitHub Container Registry, cd-web-dev.yaml för deployment av Web App, och cd-functions-dev.yaml för deployment av Function App.

Autentisering mot Azure sker via en hybridlösning: OIDC permissions (id-token: write) för säker åtkomst kombinerat med GitHub Secrets för Azure-autentiseringsuppgifter (AZURE_CLIENT_ID, AZURE_TENANT_ID, AZURE_SUBSCRIPTION_ID) och Application Insights connection string.

Deploymentflödet använder dessutom environment protection för att skilja utvecklingsmiljö från produktionsmiljö och för att säkerställa manuell kontroll där det behövs.

Varför GitHub Actions i projektet:

Det erbjuder en integrerad, säker och kostnadsfri pipeline-lösning väl anpassad för .NET-utveckling och Azure, med möjlighet att separera CI och CD för tydlighet och flexibilitet.

6.2 Docker

Docker används för att containerisera applikationen, både Blazor Server-delen och Azure Functions. Projektet använder multi-stage builds för att skapa små och optimerade images och för att separera build-miljön från runtime-miljön.

Docker löser även ett konkret problem i Azure App Service: det kringgår Oryx auto-detection-logiken, vilket ger full kontroll över runtime-versioner och dependencies. Både web-applikationen och Functions har egna Dockerfiles för att underlätta CI/CD och versionering.

Varför Docker i projektet:

Det ger reproducerbara deployments, stabil körmiljö och full kontroll över applikationens livscykel i både utveckling och hosting.

6.3 GitHub Container Registry (GHCR)

GitHub Container Registry (GHCR) används som lagringsplats för projektets Docker-images. Vid varje build pushas en ny container-image från CI-steget (ci-build.yaml), vilket gör versionering och historik enkel att följa.

För utvecklingsmiljön lagras images som public för att underlätta pull-hämtning från Azure App Service utan att behöva hantera behörigheter. GHCR är fullt integrerat med GitHub Actions, vilket gör att både build och deployment kan arbeta mot samma registry utan extern konfiguration.

Varför GHCR i projektet:

Det är en naturlig och kostnadsfri del av GitHubs ekosystem, ger enkel hantering av versionerade images och fungerar sömlöst tillsammans med Docker och App Service.

6.4 Bicep (Infrastructure as Code - IaC)

Bicep används för att definiera och hantera Azure-infrastrukturen som kod. Projektet använder en modulär struktur där resurser som App Service, Functions, Cosmos DB, Service Bus och App Configuration deklarerar i återanvändbara Bicep-moduler.

Miljön är uppdelad i separata Resource Groups för dev och prod, vilket möjliggör isolerad drift och enklare livscykelhantering. Managed Identity tilldelas direkt via Bicep, och RBAC-roller definieras explicit för att säkerställa korrekt och spårbar åtkomst i varje miljö.

Varför Bicep i projektet:

Det ger repeterbara, versionskontrollerade och dokumenterade deploymentprocesser, förenklar skapandet av nya miljöer och säkerställer att infrastrukturen alltid kan återskapas

på ett förutsägbart sätt. Terraform hade varit ett alternativ, men kunden kör idag Bicep så det var en av de "begränsningarna" jag fick jobba med i detta projekt. Bicep funkade bra.

6.5 Autentisering och secrets

Projektet använder en hybridmodell för autentisering i CI/CD, där GitHub Actions kombinerar OIDC permissions (id-token: write) med GitHub Secrets för att autentisera mot Azure och externa tjänster.

OIDC används för att generera kortlivade tokens för säker åtkomst, medan känsliga värden som AZURE_CLIENT_ID, AZURE_TENANT_ID, AZURE_SUBSCRIPTION_ID samt Application Insights connection string lagras som GitHub Secrets.

Autentisering mot GHCR sker via GITHUB_TOKEN, vilket GitHub tillhandahåller automatiskt under körning. Environment-scoped secrets används för att hålla dev- och prod-miljöer separata och förhindra oavsiktliga korsberoenden.

Varför denna lösning:

Den är säker, enkel att underhålla och följer GitHubs rekommenderade praxis för åtkomsthantering i pipelines med Azure. Jag ville använda Key Vault inom Azure-miljön men även använda GHCR för enkel deploy. Det här blev en naturlig mellanväg,

6.6 Multi-stage Docker builds

Multi-stage Docker builds används för att skapa optimerade container-images med tydlig separation mellan build- och runtime-steg.

Build-steget innehåller SDK och nödvändiga verktyg för att kompilera applikationen, medan runtime-steget endast innehåller de komponenter som behövs för körning. Detta minskar image-storleken, förbättrar säkerheten och ger snabbare deployments.

Varför multi-stage builds i projektet:

De ger små och förutsägbara images, snabbare pull-tider i App Service och en tydlig uppdelning mellan utvecklingsberoenden och produktionsmiljö.

6.7 Deployment Strategies

Projektet använder två separata deploymentstrategier beroende på komponenttyp.

Web-applikationen distribueras som Docker-containers via GHCR, där varje image taggas med en SHA-baserad versionering för spårbarhet och rollback-möjligheter.

Function App deployas via zip-deploy, vilket är den rekommenderade metoden för Azure Functions i .NET 8 isolated mode.

Utvecklingsprocessen följer en trunk-based development-modell, där kod pushas till main först efter lokal validering. Samtliga pipelines är designade för idempotent körning, vilket innebär att deployment kan triggas flera gånger utan risk för inkonsistens.

Varför denna strategi:

Den ger stabil och repeterbar leverans, tydlig versionering och låg risk vid release tack vare möjlig rollback via versionerade images.

6.8 Environment Management

Miljöhanteringen bygger på GitHub Environments, som används för både secret management och deployment approvals. Environment protection rules säkerställer att endast godkända workflows får deploya till respektive miljö.

I Azure används en dedikerad utvecklingsmiljö med Resource Group rg-examwork-dev, medan strukturen för en framtida produktionsmiljö redan är förberedd (infra/env/prod, ännu ej implementerad).

Secrets och variabler är scopade per environment, vilket säkerställer robust isolering och minimerar risken för att utvecklingsconfig används i produktion.

Varför denna lösning:

Den möjliggör en kontrollerad och säker deploymentprocess samt en tydlig separation mellan dev och framtida prod-miljöer. Det är mer en simulering av en verklig miljö än att ha något praktiskt värde idag.

7. Säkerhet

Säkerhet är en genomgående del av projektets arkitektur och utvecklingsprocess. Fokus ligger på säkra autentiseringsflöden, korrekt hantering av känslig information, rollbaserad åtkomst, robust felhantering och tillförlitlig testning. Projektet använder etablerade .NET- och Azure-mekanismer för att minimera risker och säkerställa att både data och beteenden är korrekta i alla miljöer.

7.0.1 Grundprinciper för säkerhet

Det brukar ofta vara Principle of Least Privilege (PoLP) samt Separation of Duties (SoD). Den första innebär att en användare, tjänst eller komponent endast skall ha de rättigheter som är absolut nödvändiga för att utföra sin uppgift. Inget mer. SoD är en kompletterande princip som säger att kritiska uppgifter skall delas upp mellan olika roller och system för att minska risken för missbruk, misstag eller manipulation.

7.1 Testning

Projektet använder xUnit som test-ramverk och NSubstitute för mocking av beroenden. Testningen omfattar både enhets- och integrationsnivå:

- Unit tests säkerställer affärslogik, exempelvis prisberäkningar.
- Integration tests körs via WebApplicationFactory, vilket möjliggör fullstack-testning av API och konfiguration.
- In-memory mocks (InMemoryUserService, InMemoryBookingService, InMemoryOutboxService) används för att isolera tester från externa system.
- InMemoryStorage fungerar som ett delat och simulerat datalager under testkörning.

InMemory-avsnitten kom till för att integrationstesternas tidiga iteration skapade verkliga användare på den verkliga databasen. Jag valde att skapa en mocknings-”tjänst” för att lösa den problematiken.

Varför detta i projektet:

Det ger hög testbarhet, snabb feedback och möjliggör utveckling i små steg med bibehållen systemstabilitet.

7.2 Autentisering

Autentiseringen är uppdelad i två flöden beroende på användarroll:

- Admin loggar in via Azure Entra ID, vilket ger en säker och etablerad identitetslösning baserad på OAuth 2.0 / OpenID Connect.
- Vanliga användare autentiseras via cookie-baserad inloggning med ASP.NET Core Identity-principer.

Lösenord hashas med BCrypt, och rollbaserad åtkomst kontrollerar vilka funktioner varje användartyp får nyttja. Inspector är inte färdigimplementerad men förberedd för att använda samma system som Admin.

Varför detta i projektet:

Det ger en tydlig säkerhetsuppdelning mellan privilegierade och vanliga användare, samtidigt som lösningen är enkel att underhålla och fullt kompatibel med .NET 8 och Azure.

7.3 Säkerhet i Azure

Projektet bygger på en Azure-arkitektur där säkerhet är integrerad i varje lager. Alla resurser använder Managed Identity, vilket innebär att autentisering sker utan lagrade secrets och utan behov av connection strings i kod eller konfiguration.

Åtkomst kontrolleras via RBAC (Role-Based Access Control), vilket gör att varje tjänst endast får de specifika behörigheter den behöver. Känsliga värden lagras i Key Vault, medan övriga plattformstjänster som Service Bus, Cosmos DB och App Configuration nås via identitetsbaserad åtkomst.

Nätverkssäkerheten kan i framtiden förstärkas ytterligare med private endpoints, men är för nuvarande projektsteg fullt tillräcklig i sin nuvarande form.

Hur fungerar detta i projektet:

RBAC-roller ger minsta möjliga access, Managed Identity istället för secrets, och inga övergripande roller som har för mycket tillgång. SoD är svårt att visa exempel i DevOps-funktionerna, vi har bara en enda utvecklare, annars hade vi kunnat separera deploy/godkännande. Men det vi kan göra är att separera produktionssystem och utvecklingssystem. Vi har implementerat dev och prod så där finns grunden till detta.

7.4 GDPR och Session Management

Projektet använder server-side session storage kombinerat med cookie-baserad autentisering för att säkerställa att användardata hanteras på ett säkert och GDPR-kompatibelt sätt.

Endast nödvändig information lagras i sessionen, och inga känsliga uppgifter exponeras i cookies. Lösningen är dokumenterad i ADR-010, som beskriver projektets strategi för säker sessionhantering och efterlevnad av GDPR-principer som dataminimering och åtkomstkontroll.

Hur fungerar detta i projektet:

Det säkerställer en trygg hantering av användardata och följer både tekniska och juridiska krav på modern webbapplikationsdesign, i synnerhet då alla användare har påhittade emailadresser och användardata bara för att simulera funktionaliteten.

7.5 Error Handling och Logging

Projektet använder centraliserad felhantering för både API och UI, vilket säkerställer att fel hanteras konsekvent och utan att avslöja känslig information. Samtliga fel loggas strukturerat via Application Insights, där metadata och kontext lagras för effektiv felsökning utan att kompromissa med användardata eller interna systemdetaljer.

Felmeddelanden som skickas till slutanvändaren är avsiktligt begränsade och användarvänliga, i linje med *Principle of Least Privilege* — information exponeras endast i den omfattning som behövs för att användaren ska förstå felet, inte mer.

Varför detta i projektet:

Det ger en säker, transparent och professionell felhantering samtidigt som den skyddar interna systemdetaljer och stödjer snabb felsökning under utveckling och drift.

8. Lärdomar, utveckling och slutsats

Detta avsnitt sammanfattar de viktigaste tekniska lärdomarna som framkommit under projektets utveckling. Lärdomarna speglar både arkitekturella insikter, praktiska erfarenheter och de val som visade sig mest betydelsefulla för systemets funktion, robusthet och underhållbarhet.

Det blir inget djupdyk i något av ämnena, det finns redan dokumenterat i repots `/docs/-` directory, för djupare analyser så rekommenderar jag att läsa framförallt `/journals` eller `/adr`, där utvecklar jag vissa problemställningar lite djupare.

8.1 Lärdomar av projektet

Projektet gav flera viktiga insikter kring arkitektur och praktisk systemdesign:

- *Docker som lösning på Oryx-problematiken*
Genom att containerisera applikationen kunde Oryx auto-detection i App Service helt kringgå. Det gav en stabilare och mer förutsägbar hostingmiljö.

- *Sentinel Key Pattern möjliggjorde runtime-switching*
Hot-reload av konfiguration blev centralt för live-demos och växling mellan synkront och eventdrivet läge utan omstart. Detta mönster visade sig vara en nyckelkomponent i hela arkitekturen.
- *Outbox Pattern löste dual write-problemet*
Genom transaktionssäker hantering av event kunde både databasskrivning och publicering till Service Bus garanteras utan risk för inkonsistens.
- *In-memory mocks gav effektiva tester*
Snabb, isolerad och deterministisk testning blev möjlig genom in-memory implementationer av User-, Booking- och Outbox-tjänster. Detta förbättrade testflödet avsevärt. Framförallt användes inte den riktiga databasen!
- *Extension Methods gav renare Program.cs*
Genom att flytta konfigurering till separata extension classes förbättrades läsbarheten och strukturen markant.
- *Centraliserad error handling förbättrade UX och logging*
Enhetliga felmeddelanden och strukturerad telemetri gjorde systemet lättare att felsöka och gav en bättre användarupplevelse.
- *Component extraction minskade duplicering*
Återanvändbara Razor Components (t.ex. BookingTable) reducerade kodupprepning och ökade tydligheten i UI-lagret.
- *Configuration-as-code gav flexibilitet*
Genom att flytta konfigurationsvärden till Azure App Configuration kunde systemets beteende ändras i runtime utan deploy eller restart.
- *MVP-filosofin höll projektet fokuserat*
Endast kärnfunktionalitet implementerades först, vilket undvek överdesign och möjliggjorde snabbare iterationer. Jag satt aldrig fast någon längre period på något enskilt specifikt problem!
- *Rätt verktyg för rätt uppgift*
Application Insights Workbook användes för att visualisera systemlägen och eventflöden. Det visade sig vara ett betydligt bättre verktyg för detta ändamål än traditionell logging. Custom KQL Queries var riktigt trevligt.

Sammanfattning:

Projektet visade tydligt värdet av konfigurationsstyrning, integrationsmönster och ett fokus på enkelhet och tydlighet. Flera av dessa lärdomar är direkt överförbara till både större produktionssystem och framtida projekt. En bra struktur och arkitektur och små iterativa implementationer skapade enorm kontroll och bra progress-hastighet.

8.2 Hur produkten kan utvecklas

Framtida utveckling kan sammanfattas i tre huvudkategorier: systemkritiska förbättringar, funktionella utbyggnader och kompletterande lågprioriterade förbättringar. Nedan följer en prioriterad översikt.

8.2.1 Hög prioritet – Systemkritiska förbättringar

Dessa förbättringar stärker systemets robusthet, skalbarhet och driftssäkerhet. De är direkt relevanta för en framtida produktionssättning.

- *API Management (APIM)*
Central gateway för externa API-anrop, rate limiting, skydd, versionering och analys.
- *Cosmos DB Change Feed*
Möjliggör push-baserad eventbearbetning istället för polling — särskilt värdefullt för Outbox och eventdrivna flöden.
- *Event-driven ticket expiration*
Automatisk expirering via Functions, vilket minskar beroendet på synkron logik.
- *Rate Limiting*
Skyddar API:et från missbruk och överbelastning. En viktig del av produktionsexponering.
- *Application Insights Alerts*
Automatiska notifieringar (function failures, outbox-delays, dead-letter-queue-tillstånd). Avgörande vid drift.
- *Full SSO (Single Sign-On via Entra ID)*
Enhetlig autentisering för både användare och administratörer, minskar komplexitet och förbättrar säkerhet.
- *Additional Integration Tests*
Förbättrad testtäckning för Outbox-flöden, Service Bus och Azure Functions — kritiskt för långsiktig stabilitet.

8.2.2 Mellanprioritet – Funktionella produktförbättringar

Dessa förbättringar stärker användarupplevelsen och produktens funktionella värde.

- *User Registration i UI*
Möjliggör självadministration och onboarding direkt i systemet. Jag ville inte inkludera detta i projektet då, på så sätt fick jag kontroll på trafiken/användarna..
- *Shopping Cart*
Stöd för att köpa flera biljetter innan betalning, och att välja en biljett utan att köpa.
- *Ticket Search och filtrering i admin*
Underlättar administration och felsökning.

- *QR Code Scanning*
Stöd för fysisk verifiering av biljetter i mobila enheter för Inspectors.
- *A/B-testing via feature flags*
Möjliggör gradvis utrullning av nya funktioner och datadrivet beslutsfattande.
- *Swagger/OpenAPI-dokumentation*
Gör API:et enklare att integrera med externa system och testas effektivt. Det kändes överflödigt i en MVP men är nyttigt om vi ex. kopplar ihop oss med andra regioner.
- *Domain Separation*
Förbättrar arkitekturens modularitet och långsiktiga skalbarhet.
- *Integrering av base price (zonpris) i App Configuration*
Möjliggör hot-reload av prisregler i realtid.

8.2.3 Låg prioritet – Framtida möjligheter och valfria tillägg

Dessa förbättringar är trevliga att ha men inte nödvändiga för produktionsdrift.

- *Student Verification System*
Automatisk validering av studentstatus.
- *Region-Based Pricing*
Zon- och regiondifferentierade prisregler för komplexare taxa.
- *Multi-environment staging/production*
Arkitekturen är redan förberedd; implementering sker vid behov av större driftmiljöer.

8.2.4 Sammanfattning

Genom att strukturera förbättringsförslagen efter prioritet blir det enkelt att se vad som är viktigast för systemets mognad och drift. De högst prioriterade förslagen rör robusthet och skalbarhet, medan mellanprioritet fokuserar på produktvärde. De lågprioriterade punkterna representerar framtida förbättringspotential.

8.3 Slutsats

Det här projektet har varit både utmanande och oerhört utvecklande. Jag är stolt över mina arkitekturval, min planering och genomförandet i stort. Systemet blev i princip det jag avsåg att bygga när jag startade.

Det som visade sig mest krävande var omständigheter runt projektet som jag inte kunde påverka. Inledningsvis saknades tydliga instruktioner om upplägget för examensarbetet, vilket gjorde att jag tog fram en egen plan tillsammans med mitt LIA-företag. När skolan senare tillförde nya krav och uppgifter påverkade det min tidsplanering negativt.

Mot slutet dök ytterligare en överraskning upp: i stället för att lämna in en övergripande dokumentation efter presentationen skulle både en icke-teknisk verktygspresentation och denna tekniska dokumentation lämnas in *tio dagar innan kursens slut*. Det innebar att min

sista vecka, som var avsedd för kodstädning och sammanställning av dokumentation, försvann direkt.

Ser man till dessa faktorer började projektet kännas mer som en *femveckorskurs* än en *åttaveckorskurs*, eftersom flera av veckorna gick åt till extrauppgifter, väntetider och obligatoriska moment som inte var kopplade till den egna utvecklingen.

Det gör dock inte att jag ser negativt på projektet. Tvärtom. Dessa begränsningar speglar verkligheten: ute hos kund väljer man sällan techstack själv, och förutsättningar kan förändras med kort varsel. Med det i åtanke upplever jag att jag genomfört ett realistiskt projekt och att jag uppnått mitt mål.

Det enda jag själv är mindre nöjd med är den gradvisa kodröran som uppstod under intensiv utveckling. Hinner jag innan kursen avslutas kommer jag därför städa upp koden, eftersom jag vill att slutresultatet ska representera mig väl.

Sammanfattningsvis har projektet varit mycket roligt och lärorikt. Hade jag fått göra det igen hade jag valt samma arkitektur och arbetsmetodik, men arbetat ännu mer metodiskt med att hålla koden ren under hela processen.

Trots utmaningarna känner jag att jag lämnar ifrån mig något jag är stolt över.

9. Referenser

Jag har valt att lägga hyperlänkar direkt i detta dokument, som referensmaterial ifrån denna text. Det blir lättare att läsa så, än att scrolla upp och ner mellan styckena och den här referenspunkten. Men jag ville ändå få med lite länkar till Microsofts dokumentation:

9.1 Microsoft referensmaterial

- **Microsoft Azure – Översikt över Azure-tjänster**
<https://learn.microsoft.com/azure/>
- **.NET 8**
<https://learn.microsoft.com/dotnet/core/whats-new/dotnet-8>
- **Blazor Server**
<https://learn.microsoft.com/aspnet/core/blazor/?view=aspnetcore-8.0>
- **Application Insights**
<https://learn.microsoft.com/azure/azure-monitor/app/app-insights-overview>
- **Azure Cosmos DB**
<https://learn.microsoft.com/azure/cosmos-db/>
- **Azure Service Bus**
<https://learn.microsoft.com/azure/service-bus-messaging/>
- **Azure Functions (.NET 8 isolated)**
<https://learn.microsoft.com/azure/azure-functions/dotnet-isolated-process-guide>
- **Bicep – Infrastructure as Code**
<https://learn.microsoft.com/azure/azure-resource-manager/bicep/>